

Ballista: A Web-Based Computational Platform for Projectile Dynamics and Numerical Methods Education

Technical Design Document and Multi-Year Development Blueprint

Version 1.0

Contents

1	Introduction and Vision	7
1.1	Why This Project Exists	7
1.2	Scientific Computing Relevance	7
1.3	Educational Goals	8
2	System Overview	9
2.1	Architecture	9
2.2	Module Structure	9
3	Physics Modeling Framework	11
3.1	Newtonian Formulation	11
3.2	Force Decomposition	11
3.3	Gravity	11
3.3.1	Constant Gravity	11
3.3.2	Altitude-dependent gravity	11
3.4	Drag Forces	11
3.4.1	Relative Velocity	11
3.4.2	Linear Drag	12
3.4.3	Quadratic Drag	12
3.5	Atmosphere Model	12
3.5.1	Exponential Atmosphere	12
3.5.2	ISA Model	12
3.5.3	Viscosity Model	12
3.6	3.5 Wind Interaction Model	12
3.7	3.6 Magnus Force and Spin	13
3.8	3.7 State Vector Formulation	13
3.9	3.8 Continuous-Time System Properties	14
3.10	4. Numerical Methods	14
3.10.1	4.1 Discretization	14
3.10.2	4.2 Explicit Euler	14
3.10.3	4.3 Runge–Kutta 2	14
3.10.4	4.4 RK4	14
3.10.5	4.5 Adaptive RK	14
3.10.6	4.6 Stability	15
3.10.7	4.7 Error Propagation	15
3.10.8	4.8 Symplectic Integration	15
3.10.9	4.9 Event Detection	15

3.11	5. Software Architecture	15
3.11.1	5.1 Core Engine	15
3.12	Core Simulation Interfaces (L1)	15
3.12.1	Model Interface	15
3.12.2	Stepper Interface	16
3.12.3	Solver Configuration	16
3.12.4	Integration Entry Point	16
3.13	System Design Principles	17
3.13.1	Forces System	17
3.13.2	Sink-Based Output Model	17
3.13.3	Batch Execution	17
3.13.4	Cancellation Model	17
3.13.5	Error Taxonomy	17
3.14	Extensibility Architecture	18
3.14.1	Registry Pattern	18
3.14.2	Schema-Driven UI	18
3.14.3	Versioning	18
3.15	MVC Architecture	18
3.15.1	Model	18
3.15.2	Controller	18
3.15.3	View	18
3.15.4	Key Design Constraints	18
3.16	State Management	19
3.17	Visualization System	19
3.17.1	Scene Graph	19
3.17.2	Rendering Model	19
3.17.3	Performance Constraints	19
3.18	UI System	19
3.18.1	Control Panels	19
3.18.2	Solver Lab Mode	20
3.19	Technology Choices	20
3.20	Performance Constraints	20
3.21	Task-Based Roadmap (Summary)	20
3.21.1	Structure	20
3.21.2	Phase 0: Bootstrap	20
3.21.3	Phase 1: Physics Engine	21
3.21.4	Phase 2: Numerical Methods	21
3.22	Closing Notes	21
3.23	Phase 3: UI Integration (46 Tasks)	21
3.24	Phase 4: Advanced Physics (40 Tasks)	22
3.25	Phase 5: Optimization and Inverse Problems	22
3.25.1	Inverse Problem Formulation	22
3.26	Phase 6: Monte Carlo and Uncertainty	23
3.26.1	Core Estimator	23
3.27	Phase 7: GPU and Performance	23
3.27.1	Execution Model	23
3.27.2	Performance Targets	23
3.28	Validation System	23

3.28.1 Layered Validation Architecture	23
3.28.2 Determinism Contract	24
3.29 Optimization and Inverse Problems	24
3.30 Monte Carlo Framework	24
3.31 GPU Acceleration Strategy	24
3.32 Conclusion	25

Chapter 1

Introduction and Vision

1.1 Why This Project Exists

Projectile motion with air resistance occupies a privileged position in computational physics pedagogy. It is the simplest mechanical system that is (a) nonlinear, (b) analytically intractable in its full form, (c) physically intuitive, and (d) rich enough to motivate essentially the entire numerical methods curriculum.

The quadratic-drag projectile is a canonical counterexample to closed-form solvability. With drag force:

$$\mathbf{F}_d = -\frac{1}{2}\rho C_d A \|\mathbf{v}\| \mathbf{v}$$

the system couples velocity components through speed $\|\mathbf{v}\|$, eliminating closed-form solutions in the general case.

This project builds a browser-native simulation platform in which solver behavior, truncation error, and energy drift are first-class observables.

Key Motivations

Existing tools fail in three ways:

1. **Black-box simulators:** numerical methods are hidden.
2. **Notebook fragility:** interactivity is lost.
3. **Toy architectures:** no separation of model, solver, and visualization.

1.2 Scientific Computing Relevance

The system is a miniature scientific computing stack:

- RHS abstraction $\mathbf{f}(t, \mathbf{y}; \mu)$
- ODE solver suite (Euler, RK2, RK4, adaptive methods)
- Event detection (root finding in continuous time)
- Inverse problems (shooting method)

- Monte Carlo uncertainty propagation
- GPU-ready ensemble execution

1.3 Educational Goals

The platform serves four levels:

1. Physics intuition
2. Numerical methods understanding
3. Software architecture literacy
4. Research extensibility

Chapter 2

System Overview

2.1 Architecture

The system is layered:

L5 App Shell
L4 UI Layer
L3 Visualization
L2 Runtime
L1 SolverKit
L0 Physics Engine

Dependency rule: higher layers depend only on lower layers.

2.2 Module Structure

Module	Responsibility
Engine	Physics models and forces
SolverKit	Numerical integration
Runtime	Simulation execution
Viz	Rendering
UI	Controls

Chapter 3

Physics Modeling Framework

3.1 Newtonian Formulation

The system evolves via:

$$m \frac{d^2 \mathbf{r}}{dt^2} = \sum_i \mathbf{F}_i$$

State vector:

$$\mathbf{y}(t) = (\mathbf{r}, \mathbf{v})$$

First-order form:

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y})$$

3.2 Force Decomposition

$$\mathbf{F} = \mathbf{F}_g + \mathbf{F}_{d,\text{lin}} + \mathbf{F}_{d,\text{quad}} + \mathbf{F}_M + \mathbf{F}_b + \mathbf{F}_{ext}$$

3.3 Gravity

3.3.1 Constant Gravity

$$\mathbf{F}_g = -mg\hat{\mathbf{y}}$$

3.3.2 Altitude-dependent gravity

$$g(y) = g_0 \left(\frac{R_E}{R_E + y} \right)^2$$

3.4 Drag Forces

3.4.1 Relative Velocity

$$\mathbf{v}_{rel} = \mathbf{v} - \mathbf{w}(t, \mathbf{r})$$

3.4.2 Linear Drag

$$\mathbf{F}_{d,lin} = -b\mathbf{v}_{rel}$$

3.4.3 Quadratic Drag

$$\mathbf{F}_{d,quad} = -\frac{1}{2}\rho C_d A \|\mathbf{v}_{rel}\| \mathbf{v}_{rel}$$

Reynolds number:

$$Re = \frac{\rho \|\mathbf{v}\| D}{\eta}$$

Mach number:

$$M = \frac{\|\mathbf{v}\|}{c}$$

Terminal velocity:

$$v_T = \sqrt{\frac{2mg}{\rho C_d A}}$$

3.5 Atmosphere Model

3.5.1 Exponential Atmosphere

$$\rho(y) = \rho_0 e^{-y/H}$$

3.5.2 ISA Model

$$p(y) = p_0 \left(1 - \frac{Ly}{T_0}\right)^{\frac{g}{R_s L}}$$

$$\rho = \frac{p}{R_s T}$$

3.5.3 Viscosity Model

$$\eta(T) = \eta_{ref} \left(\frac{T}{T_{ref}}\right)^{3/2} \frac{T_{ref} + S}{T + S}$$

3.6 3.5 Wind Interaction Model

Wind enters exclusively through Eq. (3.4). The `WindField` interface is $\mathbf{w}(t, \mathbf{r}) \rightarrow \mathbb{R}^d$ with implementations layered by complexity:

1. **Uniform steady:** $\mathbf{w} = (w_x, w_y)$, slider-controlled.

2. **Logarithmic boundary-layer profile:**

$$w_x(y) = \frac{u_*}{\kappa} \ln\left(\frac{y + y_r}{y_r}\right) \quad (3.13)$$

with u_* friction velocity, $\kappa = 0.41$, and roughness length $y_r \approx 0.01$ m.

Power-law alternative:

$$w_x(y) = w_{\text{ref}}(y/y_{\text{ref}})^\alpha, \quad \alpha \approx 0.14.$$

3. **Analytic gust structures:** Gaussian vortex, shear layer, sinusoidal gusts

$$w_x(t) = \bar{w} + A \sin(\Omega t + \phi).$$

4. **Stochastic gusts (Phase 6):**

$$dw' = -\frac{w'}{\tau_g} dt + \sigma_g \sqrt{\frac{2}{\tau_g}} dW_t \quad (3.14)$$

Implemented as precomputed sample paths for determinism.

5. **Gridded fields:** bilinear/bicubic interpolation from rectilinear grids.

3.7 3.6 Magnus Force and Spin

A spinning body experiences lift orthogonal to spin axis and velocity:

$$\mathbf{F}_M = \frac{1}{2} \rho C_L(S) A \|\mathbf{v}_{\text{rel}}\|^2 \frac{\hat{\boldsymbol{\omega}} \times \mathbf{v}_{\text{rel}}}{\|\hat{\boldsymbol{\omega}} \times \mathbf{v}_{\text{rel}}\|} \quad (3.15)$$

Implemented form:

$$\frac{1}{2} \rho C_L A \|\mathbf{v}_{\text{rel}}\| (\hat{\boldsymbol{\omega}} \times \mathbf{v}_{\text{rel}})$$

Spin ratio:

$$S = \frac{\omega R}{\|\mathbf{v}_{\text{rel}}\|}, \quad C_L(S) \approx \min(0.6, 1.6S) \quad (3.16)$$

Spin decay:

$$\dot{\omega} = -\omega/\tau_\omega$$

3.8 3.7 State Vector Formulation

$$\mathbf{y} = \begin{pmatrix} \mathbf{r} \\ \mathbf{v} \end{pmatrix} \in \mathbb{R}^{2d}, \quad \frac{d\mathbf{y}}{dt} = \begin{pmatrix} \mathbf{v} \\ \frac{1}{m} \sum_i \mathbf{F}_i \end{pmatrix} \quad (3.17)$$

Planar system:

$$\mathbf{y} = (x, y, v_x, v_y)^T$$

RHS (expanded)

$$\begin{aligned} \dot{x} &= v_x, & \dot{y} &= v_y \\ \dot{v}_x &= -k_d u u_x - k_m u u_y \operatorname{sgn}(\omega) \\ \dot{v}_y &= -g - k_d u u_y + k_m u u_x \operatorname{sgn}(\omega) \end{aligned} \quad (3.18)$$

3.9 3.8 Continuous-Time System Properties

Energy:

$$\frac{dE}{dt} = \mathbf{F}_{\text{aero}} \cdot \mathbf{v} \quad (3.19)$$

Drag dissipates energy; Magnus force is orthogonal to velocity.

Dimensionless group:

$$\Pi = \frac{\rho C_d A v_0^2}{2mg}$$

3.10 4. Numerical Methods

3.10.1 4.1 Discretization

$$\mathbf{y}_{k+1} = \mathbf{y}_k + h\Phi(t_k, \mathbf{y}_k)$$

Local truncation error:

$$\boldsymbol{\tau}_{k+1} = \mathcal{O}(h^{p+1})$$

Global error:

$$\|\mathbf{e}_k\| \leq \frac{C_\tau h^p}{L} (e^{L(t_k - t_0)} - 1) \quad (4.1)$$

3.10.2 4.2 Explicit Euler

$$\mathbf{y}_{k+1} = \mathbf{y}_k + hf(t_k, \mathbf{y}_k) \quad (4.2)$$

Symplectic Euler:

$$\begin{aligned} \mathbf{v}_{k+1} &= \mathbf{v}_k + ha(t_k, \mathbf{r}_k, \mathbf{v}_k) \\ \mathbf{r}_{k+1} &= \mathbf{r}_k + h\mathbf{v}_{k+1} \end{aligned} \quad (4.3)$$

3.10.3 4.3 Runge–Kutta 2

$$k_1 = f(t_k, y_k), \quad k_2 = f(t_k + c_2 h, y_k + ha_{21}k_1) \quad (4.4)$$

Order conditions:

$$b_1 + b_2 = 1, \quad b_2 c_2 = \frac{1}{2}, \quad b_2 a_{21} = \frac{1}{2} \quad (4.5)$$

3.10.4 4.4 RK4

$$\text{(Butcher tableau omitted for brevity in rendered LaTeX)} \quad (4.6)$$

3.10.5 4.5 Adaptive RK

Error estimate:

$$\delta = y_{k+1} - \hat{y}_{k+1} \quad (4.8)$$

Error norm:

$$\text{err} = \sqrt{\frac{1}{n} \sum_i \left(\frac{\delta_i}{sc_i} \right)^2} \quad (4.9)$$

Step update:

$$h_{\text{new}} = hf_s \text{err}^{-1/(\hat{p}+1)} \quad (4.10)$$

3.10.6 4.6 Stability

$$R_{\text{Euler}} = 1 + z, \quad R_{\text{RK2}} = 1 + z + \frac{z^2}{2} \quad (4.11)$$

3.10.7 4.7 Error Propagation

V-shaped error:

$$E(h) \approx C_1 h^p + C_2 \epsilon / h$$

3.10.8 4.8 Symplectic Integration

Verlet:

$$\begin{aligned} v_{k+1/2} &= v_k + \frac{h}{2} a(q_k) \\ q_{k+1} &= q_k + h v_{k+1/2} \\ v_{k+1} &= v_{k+1/2} + \frac{h}{2} a(q_{k+1}) \end{aligned} \quad (4.13)$$

3.10.9 4.9 Event Detection

Root condition:

$$g(t, y) = 0$$

3.11 5. Software Architecture**3.11.1 5.1 Core Engine**

Hot path is allocation-free.

```
interface ForceModel {
  id: string;
  accumulate(t: number, s: KinematicState, env: EnvSample,
             p: Params, outForce: Vec): void;
}
```

3.12 Core Simulation Interfaces (L1)**3.12.1 Model Interface**

The `Model` interface defines the core dynamical system representation.

```
interface Model {
  dim: number;
  channels: ChannelMeta[];

  rhs(t: number, y: F64, out: F64, ctx: EvalCtx): void;

  invariants?: InvariantSpec[];
  events?: EventSpec[];

  jacobian?(t: number, y: F64, outJ: F64): void;
  partitions?: Partitions;
}
```

Responsibilities:

- Defines system dimensionality and state structure.
- Implements RHS evaluation of ODE system.
- Optionally defines invariants, events, and Jacobians.

3.12.2 Stepper Interface

The `Stepper` defines numerical integration schemes.

```
interface Stepper {
  readonly info: {
    id: string;
    order: number;
    embeddedOrder?: number;
    fsal: boolean;
    denseOrder?: number;
    symplectic: boolean;
  };

  init(model: Model): void;

  step(t: number, y: F64, h: number, out: StepResult): void;

  interpolant?(theta: number, out: F64): void;
}
```

Properties:

- Explicit metadata for order, FSAL, and structure.
- Supports dense output interpolation.

3.12.3 Solver Configuration

```
interface SolverConfig {
  stepper: StepperId;
  h?: number;
  rtol?: number;
  atol?: number | F64;
  controller?: 'I' | 'PI';
  maxSteps: number;
  hMin?: number;
}
```

Controls adaptive integration behavior including tolerances and step size constraints.

3.12.4 Integration Entry Point

```
function integrate(
  model: Model,
  y0: F64,
  tspan: [number, number],
```



```

    cfg: SolverConfig,
    sinks: Sink[]
): SolveReport;

```

Design:

- Pull/push hybrid execution model.
- Supports cancellation and chunked execution.
- Output emitted via composable `Sink` system.

3.13 System Design Principles

3.13.1 Forces System

- Forces **accumulate additively**, never overwrite state.
- RHS evaluation uses a single reusable environment sample.
- Force ordering is deterministic (sorted by identifier).

3.13.2 Sink-Based Output Model

- `TrajectoryRecorder`: columnar storage (SoA).
- `InvariantMonitor`: energy residual tracking.
- `EventCollector`: discrete event capture.
- `StatsCollector`: solver diagnostics.

3.13.3 Batch Execution

Monte Carlo and ensemble runs:

- Emit only observables when configured.
- Optimize throughput (10^3 – 10^5 runs/sec).

3.13.4 Cancellation Model

- Cooperative scheduling via chunked integration.
- Deterministic resumption via continuation tokens.

3.13.5 Error Taxonomy

- `StepSizeUnderflow`
- `MaxStepsExceeded`
- `NonFiniteState`
- `EventLocalizationFailure`

Each error preserves last valid state and timestamp.

3.14 Extensibility Architecture

3.14.1 Registry Pattern

All extensible components register via a global registry:

```
registry.force.register(magnusForce);  
registry.stepper.register(dopri5);  
registry.model.register(planarProjectileModel);
```

3.14.2 Schema-Driven UI

- Parameter schemas automatically generate UI controls.
- Enables zero-code UI extensibility.
- Scenario specifications serialize as registry IDs + JSON.

3.14.3 Versioning

- ScenarioSpec includes `schemaVersion`.
- Migration chain ensures forward compatibility.
- Golden trajectories pin version + engine state.

3.15 MVC Architecture

3.15.1 Model

- ScenarioSpec (draft + committed)
- Trajectory / Ensemble results
- Session state

3.15.2 Controller

- SimulationSession orchestrates execution.
- Handles integration lifecycle and playback.
- Provides command API (play, pause, scrub, cancel).

3.15.3 View

- Pure functions over immutable state.
- Visualization + UI layers separated.

3.15.4 Key Design Constraints

1. Draft/committed separation for interactive editing.
2. Playback derived entirely from stored trajectory.

3.16 State Management

Store	Contents	Mutability
scenarioStore	ScenarioSpec (draft/committed)	draft mutable
resultStore	Trajectory + SolveStats	immutable
playbackStore	time, speed, mode	mutable
uiStore	layout, preferences	mutable
compareStore	pinned trajectories	append/remove

3.17 Visualization System

3.17.1 Scene Graph

- WorldLayer: physics coordinates
- TerrainLayer
- FieldLayer
- TrajectoryLayer
- GhostLayer
- ProjectileLayer
- AnnotationLayer
- HUDLayer: screen space

3.17.2 Rendering Model

- Retained-mode scene graph over Canvas2D.
- Dirty-flag optimized rendering.
- Cached trajectory geometry.

3.17.3 Performance Constraints

- 16.6 ms frame budget.
- ≤ 4 ms visualization cost steady-state.
- No per-frame allocations in render loop.

3.18 UI System

3.18.1 Control Panels

- Launch parameters (velocity, angle, spin)
- Projectile properties
- Environment configuration

- Force toggles
- Solver settings
- Simulation controls

3.18.2 Solver Lab Mode

Includes:

- Method comparison
- Convergence studies
- Stability region exploration
- Energy drift analysis

3.19 Technology Choices

Subsystem	Choice
Rendering	Canvas2D (WebGL2 Phase 7)
State	nanostores
Math rendering	KaTeX
Plots	Custom canvas + Plotly (lazy)
Numerics core	TypeScript (Float64)

3.20 Performance Constraints

- 5 ms solver budget (interactive mode)
- Transfer via ArrayBuffer only
- Worker pool parallelization
- Deterministic ensemble reduction order

3.21 Task-Based Roadmap (Summary)

3.21.1 Structure

- 278 tasks total
- 95 hours estimated implementation
- Phase-based decomposition (0–7)

3.21.2 Phase 0: Bootstrap

- Monorepo setup
- CI/CD pipeline
- Type system + linting

3.21.3 Phase 1: Physics Engine

- Forces system
- Environment models
- Projectile dynamics
- Event system

3.21.4 Phase 2: Numerical Methods

- Euler, RK2, RK4
- DOPRI5 adaptive solver
- Symplectic integrators
- Event localization

3.22 Closing Notes

This architecture prioritizes:

- Deterministic numerical computation
- Extensibility via registry + schema system
- Strict separation of model, solver, and view layers
- High-performance simulation in browser constraints

3.23 Phase 3: UI Integration (46 Tasks)

ID	Task	Est	Diff	Validation
P3.01	App shell layout: canvas center, control dock right, analysis drawer bottom	25m	E	responsive at 1280/1920/mobile; no scroll traps
P3.02	Store setup: scenario/result/playback/ui stores per §5.4	25m	M	store unit tests; snapshot immutability asserted
P3.03	SimulationSession v1: commit spec → integrate → publish result	30m	M	slider→result round trip <16 ms
P3.04	Draft/committed split with rAF-coalesced commits	20m	M	100 rapid events ⇒ ≤ 1 solve/frame
P3.05	Canvas bootstrap: DPR-aware sizing	15m	E	crisp on 2× displays
P3.06	Camera2D transforms, pan/zoom	30m	M	cursor-fixed zoom invariant

ID	Task	Est	Diff	Validation
P3.07	Auto-fit trajectory bounds	20m	M	full trajectory visible
P3.08	AxesLayer adaptive ticks	30m	M	tick density invariant across zoom
P3.09	TrajectoryLayer Path2D rendering	20m	E	deterministic parabola render
P3.10	RDP decimation on zoom	30m	H	50k pts <1ms render
P3.11	Offscreen canvas caching	25m	M	steady-state frame <4ms
P3.12	Projectile marker playback sync	20m	E	interpolant accuracy verified
P3.13	Playback clock system	30m	M	event-consistent scrub state

3.24 Phase 4: Advanced Physics (40 Tasks)

ID	Task	Est	Diff	Validation
P4.01	ISA troposphere atmosphere model	25m	M	pressure profile within tolerance
P4.02	Altitude-dependent gravity	10m	E	correct range shift sign
P4.03	Temperature-dependent Reynolds/Mach coupling	15m	E	monotonic validation checks
P4.04	Mach-dependent drag curve (PCHIP)	25m	M	transonic behavior verified
P4.05	Sports aerodynamic datasets	30m	M	literature range consistency

3.25 Phase 5: Optimization and Inverse Problems

Key system capabilities:

- Shooting method for target hitting
- Newton + line search + LM fallback
- Tangent-linear sensitivity propagation
- Basin-of-attraction visualization

3.25.1 Inverse Problem Formulation

Given target $\mathbf{r}^*$, solve:

$$F(\mathbf{u}) = \mathbf{r}_{imp}(\mathbf{u}) - \mathbf{r}^* = 0$$

Newton update:

$$\mathbf{u}_{k+1} = \mathbf{u}_k - J^{-1}F(\mathbf{u}_k)$$

3.26 Phase 6: Monte Carlo and Uncertainty

3.26.1 Core Estimator

$$\hat{Q}_N = \frac{1}{N} \sum_{i=1}^N q_i, \quad SE = \frac{\sigma_q}{\sqrt{N}}$$

Variance reduction methods:

- Antithetic variates
- Control variates
- Latin hypercube sampling
- Quasi-Monte Carlo (Sobol')

3.27 Phase 7: GPU and Performance

3.27.1 Execution Model

- CPU: adaptive RK4 / RK45
- WASM: SIMD batched RK4
- GPU: fixed-step f32 kernels (WebGPU)

3.27.2 Performance Targets

- $\geq 10^6$ trajectories/s GPU target
- <16 ms frame budget
- Zero-copy buffer pipeline where possible

3.28 Validation System

3.28.1 Layered Validation Architecture

1. Unit/property tests (symmetry, invariants)
2. Analytical comparisons (closed-form physics)
3. Convergence + conservation tests
4. Golden trajectory regression suite
5. End-to-end reproducibility tests

3.28.2 Determinism Contract

Same:

- ScenarioSpec
- Seed

$\Rightarrow$ identical result hash across runs.

3.29 Optimization and Inverse Problems

Core methods:

- Newton shooting
- Nelder-Mead
- Brent / golden-section
- Levenberg-Marquardt fallback

Conditioning issues arise near reachability envelope:

$$\det(J) \rightarrow 0$$

3.30 Monte Carlo Framework

Key extensions:

- Streaming Welford statistics
- Sobol sensitivity indices
- Importance sampling
- Control variates using analytic baselines

3.31 GPU Acceleration Strategy

- One-thread-per-trajectory model
- Fixed-step RK4 kernels
- Counter-based RNG (Philox)
- On-device reduction for observables

3.32 Conclusion

This specification defines a modular, deterministic, and extensible scientific simulation platform with:

- strict numerical correctness guarantees
- multi-layer validation architecture
- CPU/WASM/GPU execution hierarchy
- optimization and inverse problem tooling
- Monte Carlo uncertainty framework

All future extensions must preserve determinism, validation invariants, and reproducibility constraints.